

Kế hoạch: Hệ thống mời phụ huynh liên kết với học sinh

Tài liệu này mô tả chi tiết từng bước implement hệ thống mời phụ huynh. Mỗi mục ghi rõ file cần tạo/sửa, struct/interface, logic từng hàm.

Mục lục

1. Tổng quan nghiệp vụ
 2. Flow diagram
 3. Model: ParentInvitation
 4. Sửa model: ParentStudentRelation
 5. Sửa model: Notification
 6. DTO: parent_invitation_dto.go
 7. Redis keys
 8. Repository: ParentInvitationRepository
 9. Sửa Repository: ParentStudentRepository
 10. Service: ParentInvitationService
 11. Sửa Auth Service: hook Register
 12. Sửa OAuth Service: hook createOAuthUser
 13. Email templates
 14. Handler: ParentInvitationHandler
 15. Routes: parent_invitation_router.go
 16. DI wiring
 17. Database migration
 18. Cronjob: expire invitations
 19. Edge cases & bảo mật
 20. Tóm tắt files cần tạo/sửa
-

1. Tổng quan nghiệp vụ

Quy tắc

- Học sinh đăng ký **không bắt buộc** có phụ huynh
- Học sinh có thể nhập email phụ huynh (optional) — lúc đăng ký hoặc sau đó trong settings
- **Tuyệt đối không** tự động liên kết chỉ vì email trùng
- **Tuyệt đối không** tạo 2 tài khoản cùng email
- **Tuyệt đối không** cho phép truy cập dữ liệu học sinh nếu chưa được phụ huynh chấp nhận
- Một phụ huynh có thể quản lý **nhiều** học sinh
- Một học sinh có thể có **nhiều** phụ huynh/người giám hộ

- Hệ thống vẫn thu thập data tracking học tập cho mục đích nội bộ dù chưa có phụ huynh
- Báo cáo **chỉ gửi** khi có phụ huynh liên kết + đã chấp nhận

Trạng thái invitation

Status	Ý nghĩa
invited	Email chưa có tài khoản, đang chờ tạo TK
pending	Email đã có tài khoản, đang chờ phụ huynh accept/reject
accepted	Phụ huynh đã chấp nhận
rejected	Phụ huynh đã từ chối
expired	Hết hạn (7 ngày)
revoked	Bị thu hồi bởi học sinh hoặc admin

Phân biệt 2 khái niệm

- **Email nhận báo cáo** = ParentInvitation.InviteeEmail — có thể chưa có tài khoản
- **Tài khoản phụ huynh** = User với role PARENT — có đăng nhập, quản lý

2. Flow diagram

HỌC SINH ĐĂNG KÝ

1. Tạo User + UserSystemRole(STUDENT)
2. (Optional) Nhập email phụ huynh
Gọi InviteParent()

InviteParent()

1. Validate: email != student email, chưa có invitation pending
2. Check email trong hệ thống

Email ĐÃ tồn tại

```
Tạo ParentInvitation
status = "pending"
invitee_user_id = X
```

Email CHƯA tồn tại

```
Tạo ParentInvitation
status = "invited"
invitee_user_id = NULL
```

Gửi email:
"Bạn có yêu cầu
liên kết"

Tạo notification in-app

Gửi email:
"Bạn được mời tạo TK
phụ huynh trên forteX"

Link trong email:
/register?invitation=

PH ĐÃ CÓ TK

Đăng nhập → xem
invitations pending

Accept Reject

Tạo PSR Update inv
status= status=
active rejected

PH CHƯA CÓ TK

Click link → đăng ký
→ Tạo User + role PARENT
→ LinkInvitationToNewUser()
update invitee_user_id
status: invited → pending

→ Hiện invitations pending
→ PH accept/reject

PSR = ParentStudentRelation

3. Model: ParentInvitation

Tạo file: backend/internal/model/parent_invitation.go

Struct

```
type ParentInvitation struct {
    ID          uuid.UUID `gorm:"type:uuid;primaryKey;default:gen_random_uuid()" json:"id"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`

    // Học sinh gửi lời mời
    StudentUserID uuid.UUID `gorm:"type:uuid;not null;index:idx_pi_student" json:"student_user_id"`

    // Email phụ huynh được mời (có thể chưa có tài khoản)
    InviteeEmail string `gorm:"type:varchar(255);not null;index:idx_pi_email" json:"invitee_email"`
}
```

```

// NULL nếu email chưa có TK trong hệ thống
// Được set khi:
//   - Email đã tồn tại lúc tạo invitation
//   - Hoặc phụ huynh tạo TK mới sau đó (LinkInvitationToNewUser)
InviteeUserID *uuid.UUID `gorm:"type:uuid;index:idx_pi_invitee" json:"invitee_user_id"`

// Loại quan hệ: parent, guardian, grandparent
Relationship string `gorm:"type:varchar(50);not null;default:'parent'" json:"relationship"`

// Trạng thái: invited, pending, accepted, rejected, expired, revoked
Status string `gorm:"type:varchar(20);not null;default:'invited';index:idx_pi_status"`

// Token ngẫu nhiên gửi qua email, dùng để validate khi click link
// Sinh bằng crypto/rand, 32 bytes hex encode (64 chars)
Token string `gorm:"type:varchar(100);uniqueIndex;not null" json:"token"`

// Thời hạn lời mời (7 ngày kể từ khi tạo)
ExpiresAt time.Time `gorm:"not null" json:"expires_at"`

// Thời gian phụ huynh respond (accept hoặc reject)
RespondedAt *time.Time `json:"responded_at,omitempty"`

// Lời nhắn từ học sinh (optional)
Message *string `gorm:"type:text" json:"message,omitempty"`

// Relationships
Student *User `gorm:"foreignKey:StudentUserID;constraint:OnDelete:CASCADE" json:"student"`
Invitee *User `gorm:"foreignKey:InviteeUserID;constraint:OnDelete:SET NULL" json:"invitee"`
}

func (ParentInvitation) TableName() string {
    return "parent_invitations"
}

```

Constants

```

const (
    ParentInvitationStatusInvited = "invited"
    ParentInvitationStatusPending = "pending"
    ParentInvitationStatusAccepted = "accepted"
    ParentInvitationStatusRejected = "rejected"
    ParentInvitationStatusExpired = "expired"
    ParentInvitationStatusRevoked = "revoked"
)

```

Lưu ý

- Không dùng composite unique index trên (student_user_id, invitee_email) ở DB level vì cần cho phép gửi lại sau khi bị reject/expired → **check trùng ở service layer**
 - Token là unique index — mỗi invitation có 1 token duy nhất
 - InviteeUserID dùng OnDelete:SET NULL thay vì CASCADE — nếu user bị xóa, invitation vẫn còn để audit
-

4. Sửa model: ParentStudentRelation

Sửa file: backend/internal/model/parent_student_relation.go

Thêm field

```
// Link ngược về invitation đã tạo ra relation này (traceability)
InvitationID *uuid.UUID `gorm:"type:uuid" json:"invitation_id,omitempty"`
```

Không cần sửa gì khác

Model hiện tại đã có đầy đủ: - Status: pending, active, revoked - Permissions: can_view_progress, can_view_grades, ... - ConfirmedAt, ConfirmedBy - Relationship: parent, guardian, grandparent

5. Sửa model: Notification

Sửa file: backend/internal/model/notification.go

Thêm notification types vào CHECK constraint

Thêm vào danh sách check:

```
'parent_invitation'          - PH nhận được yêu cầu liên kết
'parent_invitation_accepted' - HS được thông báo PH đã chấp nhận
'parent_invitation_rejected' - HS được thông báo PH đã từ chối
```

Cách dùng

- ReferenceType = "parent_invitation" hoặc "parent_student_relation"
 - ReferenceID = invitation.ID hoặc relation.ID
-

6. DTO

Tạo file: backend/internal/dto/parent_invitation_dto.go

Request DTOs

```
// Học sinh gửi mời phụ huynh
type InviteParentRequestDto struct {
    Email      string `json:"email" validate:"required,email,max=255"`
    Relationship string `json:"relationship" validate:"required,oneof=parent guardian grandparent"`
    Message    *string `json:"message,omitempty" validate:"omitempty,max=500"`
}

// Phụ huynh respond invitation (accept hoặc reject)
type RespondInvitationRequestDto struct {
    Action string `json:"action" validate:"required,oneof=accept reject"`
}
```

Response DTOs

```
// Response sau khi gửi mời
type InviteParentResponseDto struct {
    InvitationID string `json:"invitation_id"`
    Status        string `json:"status"` // "invited" hoặc "pending"
    ParentExists  bool   `json:"parent_exists"` // email đã có TK?
    Message       string `json:"message"` // thông báo cho FE hiển thị
}

// Phụ huynh xem invitation cần respond
type PendingInvitationDto struct {
    ID            string `json:"id"`
    StudentName   string `json:"student_name"`
    StudentUsername string `json:"student_username"`
    StudentAvatar *string `json:"student_avatar,omitempty"`
    Relationship   string `json:"relationship"`
    Message       *string `json:"message,omitempty"`
    CreatedAt     string `json:"created_at"`
    ExpiresAt     string `json:"expires_at"`
}

// Học sinh xem invitation đã gửi
type SentInvitationDto struct {
    ID            string `json:"id"`
    InviteeEmail  string `json:"invitee_email"`
    InviteeName   *string `json:"invitee_name,omitempty"` // tên PH nếu đã có TK
    Status        string `json:"status"`
    Relationship   string `json:"relationship"`
    CreatedAt     string `json:"created_at"`
    RespondedAt   *string `json:"responded_at,omitempty"`
}
```

```

// Validate invitation token (public endpoint, khi click link trong email)
type ValidateInvitationTokenDto struct {
    Valid      bool    `json:"valid"`
    InvitationID string  `json:"invitation_id,omitempty"`
    StudentName string  `json:"student_name,omitempty"`
    Relationship string  `json:"relationship,omitempty"`
    HasAccount bool    `json:"has_account"` // true nếu email đã có TK
    Email      string  `json:"email,omitempty"` // email để pre-fill form đăng ký
}

```

7. Redis keys

Sửa file: backend/internal/constants/redis_keys.go

Thêm

```

const (
    PrefixParentInviteRateLimit = "parent_invite:rate"
)

// Rate limit: mỗi student tối đa 5 invitations/ngày
func KeyParentInviteRateLimit(studentID string) string {
    return fmt.Sprintf("%s:%s", PrefixParentInviteRateLimit, studentID)
}

```

Lưu ý

- Không cần lưu token vào Redis vì đã lưu trong DB (unique index)
 - Rate limit dùng Redis INCR + EXPIRE 24h
-

8. Repository: ParentInvitationRepository

Tạo file: backend/internal/repository/parent_invitation_repository.go

Interface

```

type ParentInvitationRepositoryInterface interface {
    Create(ctx context.Context, invitation *model.ParentInvitation) error
    FindByID(ctx context.Context, id uuid.UUID) (*model.ParentInvitation, error)
    FindByToken(ctx context.Context, token string) (*model.ParentInvitation, error)
    FindPendingByStudentAndEmail(ctx context.Context, studentID uuid.UUID, email string) (*model.ParentInvitation, error)
    FindPendingByInviteeUserID(ctx context.Context, userID uuid.UUID) ([]model.ParentInvitation, error)
    FindByStudentUserID(ctx context.Context, studentID uuid.UUID) ([]model.ParentInvitation, error)
}

```

```

    FindInvitedByEmail(ctx context.Context, email string) ([]model.ParentInvitation, error)
    UpdateStatus(ctx context.Context, id uuid.UUID, status string, respondedAt *time.Time) error
    UpdateInviteeUserID(ctx context.Context, id uuid.UUID, userID uuid.UUID) error
    ExpireOldInvitations(ctx context.Context) (int64, error)
}

```

Logic từng method

```
Create(ctx, invitation) → error
```

```
db.WithContext(ctx).Create(invitation).Error
```

```
FindByID(ctx, id) → (*ParentInvitation, error)
```

```
Preload("Student").Preload("Invitee")
```

```
WHERE id = ?
```

```
gorm.ErrRecordNotFound → (nil, nil)
```

```
FindByToken(ctx, token) → (*ParentInvitation, error)
```

```
Preload("Student")
```

```
WHERE token = ?
```

```
    AND status IN ('invited', 'pending')
```

```
    AND expires_at > NOW()
```

```
gorm.ErrRecordNotFound → (nil, nil)
```

```
// Chỉ tìm invitation còn hiệu lực (chưa respond, chưa hết hạn)
```

```
FindPendingByStudentAndEmail(ctx, studentID, email) → (*ParentInvitation, error)
```

```
WHERE student_user_id = ?
```

```
    AND invitee_email = ?
```

```
    AND status IN ('invited', 'pending')
```

```
gorm.ErrRecordNotFound → (nil, nil)
```

```
// Dùng để check trùng trước khi tạo invitation mới
```

```
FindPendingByInviteeUserID(ctx, userID) → ([]ParentInvitation, error)
```

```
Preload("Student")
```

```
WHERE invitee_user_id = ?
```

```
    AND status = 'pending'
```

```
Order("created_at DESC")
```

```
// Dùng cho phụ huynh xem danh sách invitations cần respond
```



```

FindByStudentUserID(ctx, studentID) → ([]ParentInvitation, error)

Preload("Invitee")
WHERE student_user_id = ?
Order("created_at DESC")

// Dùng cho học sinh xem tất cả invitations đã gửi (mọi status)

FindInvitedByEmail(ctx, email) → ([]ParentInvitation, error)

WHERE invitee_email = ?
  AND status = 'invited'

// Dùng khi phụ huynh mới tạo TK → tìm invitations chưa có TK cho email này

UpdateStatus(ctx, id, status, respondedAt) → error

updates := map[string]interface{}{
    "status": status,
}
if respondedAt != nil {
    updates["responded_at"] = respondedAt
}
db.Model(&ParentInvitation{}).Where("id = ?", id).Updates(updates)

UpdateInviteeUserID(ctx, id, userID) → error

db.Model(&ParentInvitation{}).
    Where("id = ?", id).
    Updates(map[string]interface{}{
        "invitee_user_id": userID,
        "status":          "pending", // invited → pending
    })

// Khi phụ huynh tạo TK mới, link invitation về user mới
// Status chuyển từ "invited" → "pending" (bây giờ phụ huynh có TK để respond)

ExpireOldInvitations(ctx) → (int64, error)

result := db.Model(&ParentInvitation{}).
    Where("status IN ? AND expires_at < ?",
        []string{"invited", "pending"}, time.Now()).
    Update("status", "expired")
return result.RowsAffected, result.Error

// Chạy periodic (cronjob/Asynq), dọn dẹp invitations hết hạn

```

9. Sửa Repository: ParentStudentRepository

Sửa file: backend/internal/repository/parent_student_repository.go

Thêm methods vào interface

```
// Check xem đã có relation active giữa 2 user chưa (tránh duplicate khi accept)
FindActiveByParentAndStudent(ctx context.Context, parentID, studentID uuid.UUID) (*model.ParentStudentRelation, error)

// Đếm số phụ huynh active của 1 học sinh (dùng cho logic gửi báo cáo)
CountActiveByStudentID(ctx context.Context, studentID uuid.UUID) (int64, error)
```

Logic

```
FindActiveByParentAndStudent(ctx, parentID, studentID) → (*ParentStudentRelation, error)
```

```
WHERE parent_user_id = ? AND student_user_id = ? AND status = 'active'
gorm.ErrRecordNotFound → (nil, nil)
```

```
CountActiveByStudentID(ctx, studentID) → (int64, error)
```

```
db.Model(&ParentStudentRelation{}).
    Where("student_user_id = ? AND status = ?", studentID, "active").
    Count(&count)
```

10. Service: ParentInvitationService

Tạo file: backend/internal/service/parent_invitation_service.go

Struct

```
type ParentInvitationService struct {
    cfg            *config.Config
    redisClient    *redis.Client
    invitationRepo repository.ParentInvitationRepositoryInterface
    parentStudentRepo repository.ParentStudentRepositoryInterface
    userRepo       repository.UserRepositoryInterface
}
```

Constructor

```
func NewParentInvitationService(
    cfg *config.Config,
    redisClient *redis.Client,
    invitationRepo repository.ParentInvitationRepositoryInterface,
    parentStudentRepo repository.ParentStudentRepositoryInterface,
```

```

    userRepo repository.UserRepositoryInterface,
) *ParentInvitationService

```

Hàm 1: InviteParent

```

func (s *ParentInvitationService) InviteParent(
    ctx context.Context,
    studentUserID uuid.UUID,
    req dto.InviteParentRequestDto,
) (*dto.InviteParentResponseDto, error)

```

Ai gọi: Handler khi học sinh POST /invitations/invite-parent

Logic chi tiết:

1. RATE LIMIT
 - Redis INCR trên KeyParentInviteRateLimit(studentUserID)
 - Nếu > 5 → error "bạn chỉ được gửi tối đa 5 lời mời mỗi ngày"
 - Nếu INCR == 1 → EXPIRE 24h (lần đầu trong ngày)
2. VALIDATE STUDENT
 - student = userRepo.FindUserByID(ctx, studentUserID)
 - Nếu nil → error "student not found"
 - Nếu student.Email == req.Email → error "không thể mời chính mình"
3. CHECK TRÙNG INVITATION
 - existing = invitationRepo.FindPendingByStudentAndEmail(ctx, studentUserID, req.Email)
 - Nếu existing != nil → error "đã gửi lời mời cho email này, vui lòng đợi phản hồi"
4. CHECK EXISTING RELATION (nếu email đã có TK)
 - parentUser = userRepo.FindUserByEmail(ctx, req.Email)
 - Nếu parentUser != nil:
 - existingRelation = parentStudentRepo.FindActiveByParentAndStudent(ctx, parentUser.ID, studentUserID)
 - Nếu existingRelation != nil → error "đã liên kết với phụ huynh này"
5. TẠO TOKEN
 - token = crypto/rand 32 bytes → hex encode (64 chars)
 - Hoặc dùng utils.GenerateShortCode(32) nếu đã có
6. XÁC ĐỊNH STATUS
 - Nếu parentUser != nil → status = "pending" (email đã có TK)
 - Nếu parentUser == nil → status = "invited" (email chưa có TK)
7. TẠO INVITATION


```

invitation := &model.ParentInvitation{
    StudentUserID: studentUserID,

```

```

        InviteeEmail: req.Email,
        InviteeUserID: nếu parentUser != nil thì &parentUser.ID, nil nếu không,
        Relationship: req.Relationship,
        Status:      status,
        Token:       token,
        ExpiresAt:   time.Now().Add(7 * 24 * time.Hour),
        Message:     req.Message,
    }
    invitationRepo.Create(ctx, invitation)

8. GỬI EMAIL (async, trong goroutine)
    studentName := student.FullName nếu có, student.UserName nếu không

    Nếu parentUser != nil:
        // Email đã có TK → gửi email yêu cầu liên kết
        go utils.SendParentLinkRequestEmail(cfg, req.Email, studentName, token)
    Ngược lại:
        // Email chưa có TK → gửi email mời tạo TK
        go utils.SendParentInviteEmail(cfg, req.Email, studentName, token)

9. TẠO NOTIFICATION IN-APP (nếu parentUser != nil)
    // TODO: Tạo Notification record
    // UserID:      parentUser.ID
    // Title:       "Yêu cầu liên kết phụ huynh"
    // Content:     "{studentName} muốn liên kết bạn làm {relationship}"
    // NotificationType: "parent_invitation"
    // ReferenceType:  "parent_invitation"
    // ReferenceID:   invitation.ID

10. RETURN
    return &dto.InviteParentResponseDto{
        InvitationID: invitation.ID.String(),
        Status:      status,
        ParentExists: parentUser != nil,
        Message:     tùy status,
    }

```

Hàm 2: ValidateInvitationToken

```

func (s *ParentInvitationService) ValidateInvitationToken(
    ctx context.Context,
    token string,
) (*dto.ValidateInvitationTokenDto, error)

```

Ai gọi: Handler khi user click link trong email (public endpoint)

Logic:

```
1. invitation = invitationRepo.FindByToken(ctx, token)
   - Nếu nil → return &ValidateInvitationTokenDto{Valid: false}

2. Lấy tên student từ invitation.Student (đã Preload)
   studentName := invitation.Student.FullName nếu có, UserName nếu không

3. return &ValidateInvitationTokenDto{
    Valid:      true,
    InvitationID: invitation.ID.String(),
    StudentName: studentName,
    Relationship: invitation.Relationship,
    HasAccount:  invitation.InviteeUserID != nil,
    Email:       invitation.InviteeEmail,
  }
```

Frontend dùng response này để quyết định: - HasAccount == true → redirect đến trang login rồi accept - HasAccount == false → redirect đến trang register với email pre-fill

Hàm 3: RespondToInvitation

```
func (s *ParentInvitationService) RespondToInvitation(
    ctx context.Context,
    invitationID uuid.UUID,
    parentUserID uuid.UUID,
    action string, // "accept" hoặc "reject"
) error
```

Ai gọi: Handler khi phụ huynh đã đăng nhập, respond invitation

Logic:

```
1. VALIDATE INVITATION
   invitation = invitationRepo.FindByID(ctx, invitationID)
   - Nếu nil → error "invitation not found"
   - Nếu invitation.Status != "pending" → error "invitation is not pending"
   - Nếu invitation.InviteeUserID == nil → error "invitation not linked to any user"
   - Nếu *invitation.InviteeUserID != parentUserID → error "bạn không phải người được mời"
   - Nếu invitation.ExpiresAt.Before(time.Now()):
     - invitationRepo.UpdateStatus(ctx, invitationID, "expired", nil)
     - error "invitation đã hết hạn"

2. NẾU action == "accept":
   a. Check chưa có relation active (tránh duplicate)
     existing = parentStudentRepo.FindActiveByParentAndStudent(ctx, parentUserID, invitationID)
```

Nếu existing != nil → error "đã liên kết với học sinh này"

b. Tạo ParentStudentRelation

```
now := time.Now()
confirmedBy := "parent"
relation := &model.ParentStudentRelation{
    ParentUserID:      parentUserID,
    StudentUserID:     invitation.StudentUserID,
    Relationship:       invitation.Relationship,
    Status:            model.ParentStudentStatusActive,
    CanViewProgress:   true,
    CanViewGrades:     true,
    CanViewAttendance: true,
    CanContactTeachers: true,
    CanMakePayments:   true,
    CanManageAccount:  false,
    ConfirmedAt:       &now,
    ConfirmedBy:       &confirmedBy,
    InvitationID:      &invitation.ID,
}
parentStudentRepo.CreateRelation(ctx, relation)
```

c. Update invitation

```
invitationRepo.UpdateStatus(ctx, invitationID, "accepted", &now)
```

d. Gửi notification cho student

```
// Title: "Phụ huynh đã chấp nhận liên kết"
// Content: "{parentName} đã chấp nhận làm {relationship} của bạn"
// Type: "parent_invitation_accepted"
// ReferenceType: "parent_student_relation"
// ReferenceID: relation.ID
```

3. Nếu action == "reject":

a. Update invitation

```
now := time.Now()
invitationRepo.UpdateStatus(ctx, invitationID, "rejected", &now)
```

b. Gửi notification cho student

```
// Title: "Yêu cầu liên kết bị từ chối"
// Content: "Phụ huynh đã từ chối yêu cầu liên kết"
// Type: "parent_invitation_rejected"
// ReferenceType: "parent_invitation"
// ReferenceID: invitation.ID
```

Hàm 4: GetPendingInvitations

```
func (s *ParentInvitationService) GetPendingInvitations(  
    ctx context.Context,  
    parentUserID uuid.UUID,  
) ([]dto.PendingInvitationDto, error)
```

Ai gọi: Handler khi phụ huynh xem invitations cần respond

Logic:

1. invitations = invitationRepo.FindPendingByInviteeUserID(ctx, parentUserID)
 2. Map thành []PendingInvitationDto:
 - StudentName = inv.Student.FullName || inv.Student.UserName
 - StudentUsername = inv.Student.UserName
 - StudentAvatar = inv.Student.AvatarURL
 - ...format timestamps thành RFC3339 string
 3. Return
-

Hàm 5: GetSentInvitations

```
func (s *ParentInvitationService) GetSentInvitations(  
    ctx context.Context,  
    studentUserID uuid.UUID,  
) ([]dto.SentInvitationDto, error)
```

Ai gọi: Handler khi học sinh xem invitations đã gửi

Logic:

1. invitations = invitationRepo.FindByStudentUserID(ctx, studentUserID)
 2. Map thành []SentInvitationDto:
 - InviteeEmail = inv.InviteeEmail
 - InviteeName = inv.Invitee.FullName nếu Invitee != nil
 - ...format timestamps
 3. Return
-

Hàm 6: RevokeInvitation

```
func (s *ParentInvitationService) RevokeInvitation(  
    ctx context.Context,  
    invitationID uuid.UUID,  
    studentUserID uuid.UUID,  
) error
```

Ai gọi: Handler khi học sinh thu hồi lời mời

Logic:

1. invitation = invitationRepo.FindByID(ctx, invitationID)
 - Nếu nil → error "invitation not found"
 - Nếu invitation.StudentUserID != studentUserID → error "bạn không phải người gửi invitation"
 - Nếu invitation.Status NOT IN ("invited", "pending") → error "chỉ có thể thu hồi invitation"
 2. invitationRepo.UpdateStatus(ctx, invitationID, "revoked", nil)
-

Hàm 7: LinkInvitationToNewUser

```
func (s *ParentInvitationService) LinkInvitationToNewUser(
    ctx context.Context,
    email string,
    userID uuid.UUID,
) error
```

Ai gọi: Auth Service sau khi phụ huynh mới tạo TK (Register hoặc OAuth)

QUAN TRỌNG: Hàm này **KHÔNG** auto-accept. Chỉ link InviteeUserID và chuyển status invited → pending.

Logic:

1. Tìm tất cả invitation cho email này mà status = "invited"


```
invitations = invitationRepo.FindInvitedByEmail(ctx, email)
```
 2. Nếu len(invitations) == 0 → return nil (không có invitation nào)
 3. Với mỗi invitation:


```
invitationRepo.UpdateInviteeUserID(ctx, inv.ID, userID)
// Hàm này set invitee_user_id = userID VÀ status = 'pending'
```
 4. Gửi notification cho user mỗi:


```
// Title: "Bạn có lời mời liên kết phụ huynh"
// Content: "Bạn có {len(invitations)} yêu cầu liên kết từ học sinh đang chờ"
// Type: "parent_invitation"
```
 5. Return nil
-

11. Sửa Auth Service

Sửa file: backend/internal/service/auth_service.go

Thêm dependency

```
type AuthService struct {  
    // ...existing fields...  
    parentInvitationService *ParentInvitationService // thêm mới  
}
```

Hoặc nếu muốn tránh circular dependency: inject interface thay vì concrete type.

Sửa hàm Register()

Sau khi tạo User + UserSystemRole thành công (sau dòng 263):

```
// Sau khi tạo user thành công, check xem có invitation nào cho email này không  
// Nếu có phụ huynh nào mời email này + link invitation về user mới  
// KHÔNG auto-accept, chỉ chuyển status invited + pending  
if s.parentInvitationService != nil {  
    go func() {  
        bgCtx := context.Background()  
        if err := s.parentInvitationService.LinkInvitationToNewUser(bgCtx, user.Email, user.ID); err != nil {  
            log.Printf("[WARN] Failed to link invitations for user %s: %v", user.Email, err)  
        }  
    }()  
}
```

Lưu ý

- Dùng `go func()` vì đây là side effect, không cần block Register flow
- Dùng `context.Background()` vì goroutine chạy ngoài request context
- Chạy cho MỌI role đăng ký, không chỉ PARENT — vì email phụ huynh có thể đăng ký bất kỳ role nào rồi thêm profile PARENT sau

12. Sửa OAuth Service

Sửa file: `backend/internal/service/oauth_service.go`

Sửa hàm `createOAuthUser()`

Tương tự Auth Service, sau khi tạo User + UserSystemRole + UserOAuth-Provider:

```
// Link invitations nếu có  
if s.parentInvitationService != nil {  
    go func() {  
        bgCtx := context.Background()  
        _ = s.parentInvitationService.LinkInvitationToNewUser(bgCtx, *ghUser.Email, user.ID)  
    }()  
}
```

```
    }()  
}
```

Thêm dependency

Thêm `parentInvitationService` vào `OAuthService` struct và constructor.

13. Email templates

Sửa file: `backend/internal/utils/email.go`

Email 1: `SendParentInviteEmail` — Mời tạo tài khoản

```
func SendParentInviteEmail(cfg *config.Config, toEmail, studentName, token string) error
```

Khi email chưa có tài khoản trong hệ thống.

Nội dung email:

Subject: "{studentName} mời bạn làm phụ huynh trên forteX"

Body:

- Chào bạn,
- {studentName} đã mời bạn làm phụ huynh/người giám hộ trên hệ thống forteX.
- Khi chấp nhận, bạn có thể theo dõi tiến độ học tập, điểm số và hoạt động của con.
- [Nút: Tạo tài khoản phụ huynh]
→ Link: {cfg.FrontendURL}/register?invitation={token}
- Lời mời này hết hạn sau 7 ngày.
- Nếu bạn không biết {studentName}, vui lòng bỏ qua email này.

Email 2: `SendParentLinkRequestEmail` — Yêu cầu liên kết

```
func SendParentLinkRequestEmail(cfg *config.Config, toEmail, studentName, token string) error
```

Khi email đã có tài khoản trong hệ thống.

Nội dung email:

Subject: "{studentName} muốn liên kết phụ huynh với bạn"

Body:

- Chào bạn,
- {studentName} đã gửi yêu cầu liên kết bạn làm phụ huynh/người giám hộ.
- Khi chấp nhận, bạn có thể:
 - Xem tiến độ học tập
 - Xem điểm số và bài kiểm tra
 - Nhận báo cáo định kỳ
- [Nút: Xem yêu cầu]

- Link: {cfg.FrontendURL}/invitations?token={token}
- Lời mời này hết hạn sau 7 ngày.
- Nếu bạn không biết {studentName}, vui lòng bỏ qua email này.

Format email

Theo pattern hiện tại của project (HTML table layout, style inline, brand “fortex”). Copy template từ `SendRegisterOTP` và thay nội dung.

14. Handler: ParentInvitationHandler

Tạo file: `backend/internal/handler/parent_invitation_handler.go`

Struct

```
type ParentInvitationHandler struct {
    invitationService *service.ParentInvitationService
}

func NewParentInvitationHandler(svc *service.ParentInvitationService) *ParentInvitationHandler {
    return &ParentInvitationHandler{invitationService: svc}
}
```

Handler 1: InviteParent

Route: POST /invitations/invite-parent

Auth: Required (role: bất kỳ, service sẽ check student)

```
func (h *ParentInvitationHandler) InviteParent(c *fiber.Ctx) error {
    // 1. Lấy student user ID từ JWT
    studentUserID := c.Locals("user_id").(uuid.UUID)

    // 2. Parse và validate request body
    var req dto.InviteParentRequestDto
    if err := c.BodyParser(&req); err != nil {
        return c.Status(400).JSON(fiber.Map{"message": "Invalid request body"})
    }
    if errors := utils.ValidateStruct(req); len(errors) > 0 {
        return c.Status(400).JSON(fiber.Map{"message": "Validation failed", "errors": errors})
    }

    // 3. Gọi service
    result, err := h.invitationService.InviteParent(c.Context(), studentUserID, req)
    if err != nil {
        return c.Status(400).JSON(fiber.Map{"message": err.Error()})
    }
}
```

```

    return c.Status(201).JSON(fiber.Map{"message": "Invitation sent", "data": result})
}

```

Handler 2: ValidateToken

Route: GET /invitations/validate/:token

Auth: NOT required (public)

```

func (h *ParentInvitationHandler) ValidateToken(c *fiber.Ctx) error {
    token := c.Params("token")
    result, err := h.invitationService.ValidateInvitationToken(c.Context(), token)
    if err != nil {
        return c.Status(400).JSON(fiber.Map{"message": err.Error()})
    }
    return c.JSON(fiber.Map{"data": result})
}

```

Handler 3: RespondToInvitation

Route: POST /invitations/:id/respond

Auth: Required

Body: {"action": "accept"} hoặc {"action": "reject"}

```

func (h *ParentInvitationHandler) RespondToInvitation(c *fiber.Ctx) error {
    parentUserID := c.Locals("user_id").(uuid.UUID)
    invitationID, err := uuid.Parse(c.Params("id"))
    if err != nil {
        return c.Status(400).JSON(fiber.Map{"message": "Invalid invitation ID"})
    }

    var req dto.RespondInvitationRequestDto
    if err := c.BodyParser(&req); err != nil {
        return c.Status(400).JSON(fiber.Map{"message": "Invalid request body"})
    }
    if errors := utils.ValidateStruct(req); len(errors) > 0 {
        return c.Status(400).JSON(fiber.Map{"message": "Validation failed", "errors": errors})
    }

    if err := h.invitationService.RespondToInvitation(c.Context(), invitationID, parentUserID, req.Action); err != nil {
        return c.Status(400).JSON(fiber.Map{"message": err.Error()})
    }

    return c.JSON(fiber.Map{"message": "Invitation " + req.Action + "ed"})
}

```

Handler 4: GetPendingInvitations

Route: GET /invitations/pending

Auth: Required

```
func (h *ParentInvitationHandler) GetPendingInvitations(c *fiber.Ctx) error {
    parentUserID := c.Locals("user_id").(uuid.UUID)
    result, err := h.invitationService.GetPendingInvitations(c.Context(), parentUserID)
    if err != nil {
        return c.Status(500).JSON(fiber.Map{"message": err.Error()})
    }
    return c.JSON(fiber.Map{"data": result})
}
```

Handler 5: GetSentInvitations

Route: GET /invitations/sent

Auth: Required

```
func (h *ParentInvitationHandler) GetSentInvitations(c *fiber.Ctx) error {
    studentUserID := c.Locals("user_id").(uuid.UUID)
    result, err := h.invitationService.GetSentInvitations(c.Context(), studentUserID)
    if err != nil {
        return c.Status(500).JSON(fiber.Map{"message": err.Error()})
    }
    return c.JSON(fiber.Map{"data": result})
}
```

Handler 6: RevokeInvitation

Route: DELETE /invitations/:id

Auth: Required

```
func (h *ParentInvitationHandler) RevokeInvitation(c *fiber.Ctx) error {
    studentUserID := c.Locals("user_id").(uuid.UUID)
    invitationID, err := uuid.Parse(c.Params("id"))
    if err != nil {
        return c.Status(400).JSON(fiber.Map{"message": "Invalid invitation ID"})
    }

    if err := h.invitationService.RevokeInvitation(c.Context(), invitationID, studentUserID) {
        return c.Status(400).JSON(fiber.Map{"message": err.Error()})
    }

    return c.JSON(fiber.Map{"message": "Invitation revoked"})
}
```

15. Routes

Tạo file: backend/internal/router/parent_invitation_router.go

```
func SetupParentInvitationRoutes(  
    api fiber.Router,  
    cfg *config.Config,  
    handler *handler.ParentInvitationHandler,  
    redisClient *redis.Client,  
) {  
    inv := api.Group("/invitations")  
  
    // ===== Public endpoint =====  
    // Validate token từ email link (không cần đăng nhập)  
    inv.Get("/validate/:token", handler.ValidateToken)  
  
    // ===== Protected endpoints =====  
    inv.Use(middleware.AuthMiddleware(cfg, redisClient))  
  
    // Học sinh gửi mời phụ huynh  
    inv.Post("/invite-parent", handler.InviteParent)  
  
    // Học sinh xem invitations đã gửi  
    inv.Get("/sent", handler.GetSentInvitations)  
  
    // Phụ huynh xem invitations pending  
    inv.Get("/pending", handler.GetPendingInvitations)  
  
    // Phụ huynh accept/reject  
    inv.Post("/:id/respond", handler.RespondToInvitation)  
  
    // Học sinh thu hồi invitation  
    inv.Delete("/:id", handler.RevokeInvitation)  
}
```

16. DI wiring

backend/internal/app/repositories.go

Thêm vào struct Repositories:

ParentInvitation *repository.ParentInvitationRepository

Thêm vào InitRepositories():

ParentInvitation: repository.NewParentInvitationRepository(db),

`backend/internal/app/services.go`

Thêm vào struct `Services`:

```
ParentInvitation *service.ParentInvitationService
```

Thêm vào `InitServices()`:

```
parentInvitationSvc := service.NewParentInvitationService(  
    resources.Config,  
    resources.Redis,  
    repos.ParentInvitation,  
    repos.ParentStudent,  
    repos.User,  
)
```

```
// Inject vào AuthService (nếu cần hook Register)
```

```
// Hoặc set sau khi tạo authService:
```

```
// authService.SetParentInvitationService(parentInvitationSvc)
```

Thêm vào return `&Services{}`:

```
ParentInvitation: parentInvitationSvc,
```

`backend/internal/app/handlers.go`

Thêm vào struct `Handlers`:

```
ParentInvitation *handler.ParentInvitationHandler
```

`Init`:

```
ParentInvitation: handler.NewParentInvitationHandler(services.ParentInvitation),
```

`backend/internal/app/app.go` hoặc `router/route.go`

Wire routes:

```
router.SetupParentInvitationRoutes(api, cfg, handlers.ParentInvitation, redis)
```

17. Database migration

Sửa file: `backend/internal/database/postgres.go`

Thêm vào auto-migration list (sau `&model.ParentStudentRelation{}`):

```
&model.ParentInvitation{}
```

18. Cronjob

Expire invitations hết hạn

Dùng Asynq (đã có trong project) hoặc goroutine chạy mỗi giờ:

```
// Mỗi giờ:
affected, err := invitationRepo.ExpireOldInvitations(ctx)
if affected > 0 {
    log.Printf("Expired %d parent invitations", affected)
}
```

Nơi đặt

Có thể tạo Asynq task type mới hoặc thêm vào scheduler hiện có.

19. Edge cases & bảo mật

#	Case	Xử lý
1	Student mời chính mình	Validate email != student.Email
2	Gửi trùng invitation pending	FindPendingByStudentAndEmail check trước khi tạo
3	Đã linked rồi mời lại	FindActiveByParentAndStudent check
4	Gửi lại sau khi bị reject	Cho phép — FindPending chỉ check status invited/pending
5	Token hết hạn	FindByToken filter expires_at > NOW()
6	Token replay (dùng lại sau accept)	Sau respond, status thay đổi → FindByToken filter status
7	PH accept nhưng không có role PARENT	Vẫn cho accept — PH có thể thêm role PARENT sau qua CreateProfile
8	Rate limit spam	Redis INCR + EXPIRE, max 5/ngày/student
9	Trẻ vị thành niên	Data tracking vẫn collect, report chỉ gửi khi có parent accepted
10	Parent muốn revoke sau khi accept	Endpoint riêng: cập nhật ParentStudentRelation.Status = 'revoked'
11	Student muốn remove parent	Endpoint riêng: cập nhật ParentStudentRelation.Status = 'revoked'

#	Case	Xử lý
12	Email injection trong parent email	Validate email format ở DTO layer (tag <code>validate:"email"</code>)
13	Token brute force	Token 64 chars hex (256 bits entropy) — không thể brute force
14	PH tạo TK bằng OAuth (Google/GitHub)	LinkInvitationToNewUser cũng chạy trong OAuth flow
15	2 student mời cùng 1 email PH	Cho phép — mỗi student tạo invitation riêng, PH accept/reject từng cái

20. Tóm tắt files

Files cần TẠO MỚI

File	Nội dung
model/parent_invitation.go	Struct + constants
dto/parent_invitation_dto.go	2 request + 4 response DTOs
repository/parent_invitation_repository.go	Interface + 9 methods
service/parent_invitation_service.go	7 methods
handler/parent_invitation_handler.go	6 handlers
router/parent_invitation_router.go	Route setup

Files cần SỬA

File	Nội dung sửa
model/parent_student_relation.go	+1 field InvitationID
model/notification.go	+3 notification types trong CHECK constraint
utils/email.go	+2 email templates
constants/redis_keys.go	+1 rate limit key
repository/parent_student_repository.go	+1 repository interface + implementation
service/auth_service.go	Hook Register → LinkInvitationToNewUser
service/oauth_service.go	Hook createOAuthUser → LinkInvitationToNewUser
database/postgres.go	+1 model trong auto-migration
app/repositories.go	+1 repo
app/services.go	+1 service
app/handlers.go	+1 handler
app/app.go hoặc router/route.go	Wire routes

Thứ tự implement khuyến nghị

- | | |
|----------------|---|
| 1. Model | → parent_invitation.go |
| 2. DTO | → parent_invitation_dto.go |
| 3. Migration | → thêm vào postgres.go |
| 4. Repository | → parent_invitation_repository.go |
| 5. Sửa repo | → parent_student_repository.go (+2 methods) |
| 6. Redis keys | → redis_keys.go |
| 7. Email | → email.go (+2 templates) |
| 8. Service | → parent_invitation_service.go |
| 9. Sửa service | → auth_service.go, oauth_service.go (hook) |
| 10. Handler | → parent_invitation_handler.go |
| 11. Routes | → parent_invitation_router.go |
| 12. DI wiring | → repositories.go, services.go, handlers.go, app.go |
| 13. Cronjob | → expire task |
| 14. Test | → go test |